

|                                                                                                                                                           |                                                                                       |                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------|
|  <b>U.PORTO</b><br>FEUP FACULDADE DE ENGENHARIA<br>UNIVERSIDADE DO PORTO | MECD –Master Degree in Data Science and Engineering<br><b>Artificial Intelligence</b> | 2025/2026<br>(1st Year)<br>2 <sup>nd</sup> Semester |
| TEACHER: Luís Paulo Reis                                                                                                                                  |                                                                                       |                                                     |

# Practical Work

## Heuristic Search Methods for Problem Solving

### Adversarial Search Methods for Games

### Optimization Methods/Meta-Heuristics

#### Topic 1: Heuristic Search Methods for One Player Solitaire Games

A solitaire game is characterized by the type of board and pieces, the rules of movement of the pieces (possible operators/moves), and the conditions for ending the game with defeat (impossibility to solve, maximum number of moves reached, time limit reached) or victory (solitaire solved), together with the respective score. Typically, in the event of a win, a score is awarded depending on the number of moves, resources spent, bonuses collected, and/or time spent.

In addition to implementing a solitaire game for a human player, the program must be able to solve different versions/levels of this game, using appropriate search methods, focusing on the comparison between uninformed search methods (breadth-first search, depth-first search, iterative deepening, uniform cost) and heuristic search methods (greedy search, A\* Algorithm, Weighted A\*, ...), all using with different heuristic functions. The algorithms employed should be compared with several criteria, with emphasis on the quality of the solution obtained, the number of states generated, maximum memory used, and time spent to obtain the solution. The work may also include the use of Metaheuristics and its comparison with the use of search methods. All games, if it is possible, should have variable board sizes and a set of puzzles to solve with different difficulty levels. The application should be able to read the puzzle state (board position) from text files and to store the results (including the puzzle solution, time and memory taken for each algorithm, final results, etc.) also in text files.

Students should focus on firstly developing a simple version of the game with a small size board, the algorithms and testing that everything works (algorithms are able to solve the simple small puzzles), before proceeding to more complex and bigger scenarios.

The application should have a graphical user interface to show the evolution of the board and interact with the user/player. It should allow a game mode in which the PC solves the solitaire alone using the algorithms implemented and configuration selected by the user, giving the details of the solution, and time and memory used to solve the puzzle. The application should also allow a Human game mode in which the user can interactively solve the game while asking the PC for “hints” (next move).

#### Topic 2: Adversarial Search Methods for Two-Player Board Games

A board game is characterized by the type of board and pieces, the rules of movement of the pieces (operators), and the finishing conditions of the game with the respective score (typically win for player 1, win for player 2 or draw). In this work, the aim is to implement a game for two players and solve different versions of this game, using the Minimax search algorithm with  $\alpha\beta$  cuts and its variants and with Monte Carlo Tree Search and its variants.

Human-human, human-computer, and computer-computer game modes should be developed, where the computer should exhibit different skills (levels of difficulty). Computer performance should be compared regarding the different skills (e.g., hard, medium, easy), corresponding to different evaluation functions, different depth levels of Minimax, different successor generation ordering, and/or variants of the Minimax algorithm. Monte Carlo Tree Search with different configurations should also be used. Emphasis should

be placed on the analysis of the results of the computer players (wins, draws, losses, and other quality parameters, such as the number of plays to obtain the win/loss) and the average time spent and memory used to obtain the computer moves. All games, if it is possible, should have different versions with variable board sizes. The application should be able to read the game state (board position) from text files and to store the results (including the moves done by each player, time taken, final results, etc) also in text files.

Students should focus on firstly developing a simple version of the game with a small size board, the algorithms and testing that everything works (algorithms are able to show intelligent/correct moves in simple game situations), before proceeding to more complex and bigger scenarios.

The application to be developed must have a proper graphical user interface, to show the evolution of the board and interact with the user/player. Different game modes must be included, as explained above, allowing the selection of the game mode, type of each player, algorithms used and skills of each of the computer players. The application should enable selecting different levels and heuristics for each of the computer players to play against each other. The work should also enable providing human players with the next movement suggestion “hints”.

### **Topic 3: Metaheuristics for Optimization/Decision Problems**

An optimization problem is characterized by the existence of a (typically large) set of possible solutions, comparable to each other, of which one or more are considered (globally) optimal solutions. Depending on the specific problem, an evaluation function allows you to establish this comparison between solutions. In many of these problems, it is virtually impossible to find the optimal solution or to ensure that the solution found is optimal, and, as such, the goal is to try to find a locally optimal solution that maximizes/minimizes a given evaluation function to the extent possible.

In this work, the aim is to implement a system to solve an optimization problem, using different algorithms or meta-heuristics, such as hill-climbing, simulated annealing, tabu search, and genetic algorithms. Other algorithms or variations of these algorithms may also be included. Multiple instances, with different sizes and complexities, of the chosen problem, must be solved, and the results obtained by each algorithm must be compared. Different parameterizations of the algorithms should be tested and compared, in terms of the average quality of the solution obtained and the average time spent to obtain the solutions. In the algorithm comparison try to make all the algorithms take similar time in achieving the solution and compare the quality of the solutions (for example making 20x more iterations for a tabu search algorithm than for a genetic algorithm with population of 20). All problems should have different sizes/difficulties to solve. The application should be able to read the problems from text files and to store the results (including the path to the solution, quality, time, etc) also in text files.

Students should focus on firstly developing a simple version of the application with a small simple problem, the algorithms and testing that everything works (all algorithms achieve the optimal solution for the simple problem), before proceeding to more complex and bigger scenarios.

The application to be developed should have an appropriate visualization in graphic mode, to show the evolution of the solution and its quality obtained along the way. The application should also enable to analyse the final (i.e. local optimal) solution(s), and to interact with the user. You should provide in the interface the means for the selection and parameterization of the algorithms and the selection of the instance of the problem to be solved.

## **Programming Language**

Any programming language and development system can be used, including, at the language level, Python, C++, Java, C#, Prolog, among others. The choice of language and development environment to be used is the entire responsibility of the students. However, the use of Python is typically preferred. Students may use any library that may be useful for the work (NumPy, SciPy, PyGame, Matplotlib, among many others),

## **Groups**

Groups must be composed of 3 students (exceptionally 2).

## Checkpoint

Each group must submit in Moodle by March 6<sup>th</sup> (8-11am) a brief presentation (max. 5 slides), in PDF format, which will be used in the classes (Saturday 7<sup>th</sup> 8h30-13h30) to analyze, together with the teacher, the progress of the work. The presentation should contain (1) a specification of the work to be performed (definition of the game or optimization problem to be solved), (2) related work with references to works found in a bibliographic search (articles, web pages, and/or source code), (3) formulation of the problem as a search problem (state representation, initial state, objective test, operators (names, preconditions, effects, and costs), heuristics/evaluation function) or optimization problem (solution representation, neighborhood/mutation and crossover functions, hard constraints, evaluation functions), and (4) implementation work already carried out (programming language, development environment, data structures, file structures, among others).

## Final Delivery

Each group must submit in Moodle the following files: a presentation (max. 20 slides), in PPT format and the implemented code, properly commented, including a “readme” file with instructions on how to compile, run and use the program. To improve the work, the students may submit a paper/report (in IEEE format) or a new PPT and video describing and demonstrating the work with possible improvements (max 5 minutes). Based on the submitted presentation, the students must carry out a presentation with demonstration (about 10 minutes + 5 minutes for discussion) of the work, in the final evaluation class.

The presentation should contain: (1) specification of the work performed (definition of the game or optimization problem to be solved), (2) related work with references to works found in a bibliographic search (articles, web pages and/or source code), (3) formulation of the problem as a search problem (state representation, initial state, objective test, operators (names, preconditions, effects and costs), heuristics/evaluation function) or optimization problem (solution representation, neighborhood/mutation and crossover functions, rigid constraints, evaluation functions), (4) implementation details (programming language, development environment, data structures, file structure, among others), (5) the approach (heuristics, evaluation functions, operators, ...) and (6) algorithms implemented (search algorithms, minimax, metaheuristics), as well as (7) experimental results, using appropriate tables/plots and comparing the various methods, heuristics, algorithms and respective parameterizations for different scenarios/problems of different sizes and complexities. The presentation shall include also, introduction and conclusions and the references consulted and materials used (software, websites, scientific articles, ...).

## Suggested Problems

### Topic 1: One Player Solitaire Games

1A) Atomix

[https://en.wikipedia.org/wiki/Atomix\\_\(video\\_game\)](https://en.wikipedia.org/wiki/Atomix_(video_game))

1B) Space Block – Roll the Block -

<https://play.google.com/store/apps/details?id=com.zawor.spaceblock&hl=en&gl=US>

### Topic 2: Two-Player Adversarial Games

2A) Emergo (smaller, flexible dimension boards)

[https://en.wikipedia.org/wiki/Emergo\\_\(board\\_game\)](https://en.wikipedia.org/wiki/Emergo_(board_game))

<https://www.mindsports.nl/index.php/dagaz/796-emergo-7x7>

2B) Bound

<https://boardgamegeek.com/boardgame/375975/bound>

### Topic 3: Optimization Problems

3A) Flow Shop, Job Shop or Open Shop Scheduling Problems

<http://mistic.heig-vd.ch/taillard/articles.dir/Taillard1993EJOR.pdf>

<http://mistic.heig-vd.ch/taillard/problemes.dir/ordonnancement.dir/ordonnancement.html>

3B) Google Hash Code 2021 - Even more pizza

[https://github.com/pierreavn/google-hashcode-archive/blob/main/archive/2021/practice/hashcode\\_2021\\_practice\\_round.pdf](https://github.com/pierreavn/google-hashcode-archive/blob/main/archive/2021/practice/hashcode_2021_practice_round.pdf)

3C) Google Hash Code 2018 - Self-driving rides

[https://github.com/pierreavn/google-hashcode-archive/blob/main/archive/2018/qualification/hashcode\\_2018\\_qualification\\_round.pdf](https://github.com/pierreavn/google-hashcode-archive/blob/main/archive/2018/qualification/hashcode_2018_qualification_round.pdf)

3D) Google Hash Code 2021 - Traffic signaling

<https://github.com/pierreavn/google-hashcode-archive/tree/main/archive/2021/qualification>

3E) Google Hash Code 2018 - City Plan

[https://github.com/pierreavn/google-hashcode-archive/blob/main/archive/2018/final/hashcode\\_2018\\_final\\_round.pdf](https://github.com/pierreavn/google-hashcode-archive/blob/main/archive/2018/final/hashcode_2018_final_round.pdf)